Numerical Quantum Mechanics: Scattering problem

Young-Ho Song (IRIS, IBS)

IRIS 중이온가속기연구소
Institute for Rare Isotope Science

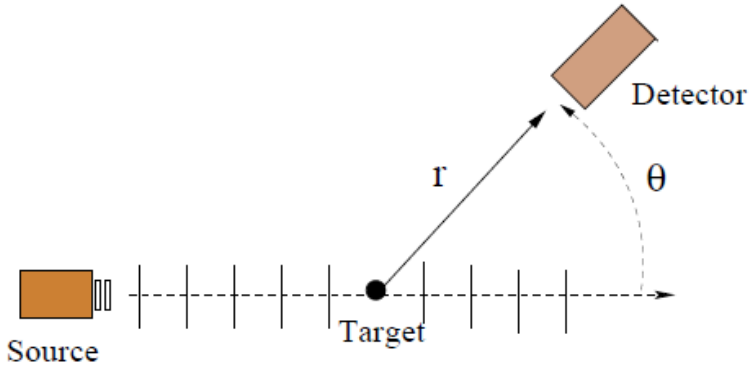
2026.03.31. Yonsei University

Introduction

- Why Scattering(Reaction)?
  - We cannot directly observe quantum particles, nuclei, atoms
  - →To get information on the interaction between quantum particles, nuclei, atoms.
  - →To get information on the structure of quantum particles, nuclei, atoms.
  - →To measure reaction rates of particular interest.

    (stars, nuclear reactors, production of radioactive isotopes)

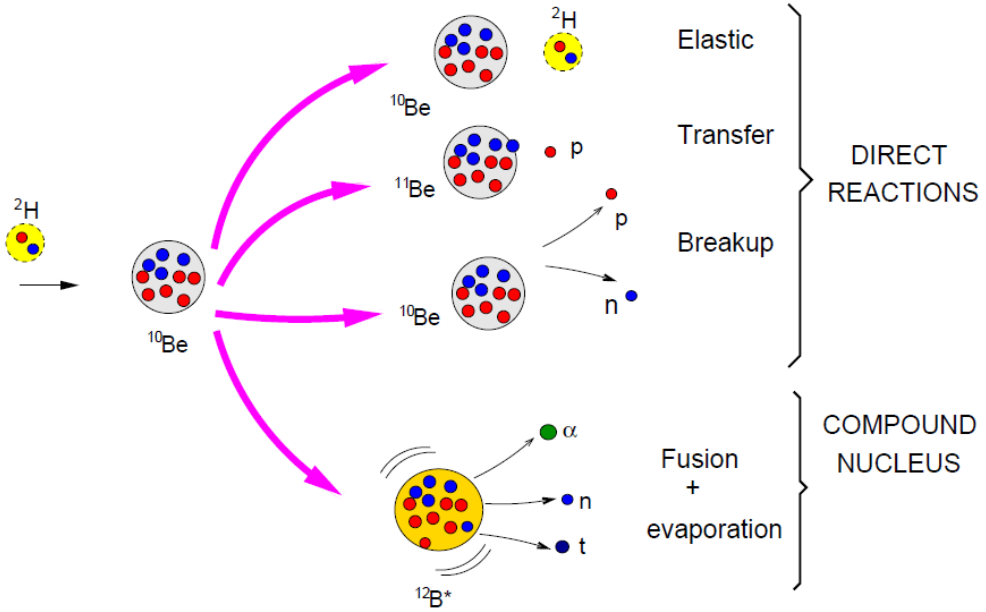
- How experiments are done?
  - → Measure the distribution of products.
  - → At far away from the reaction spot.



Types of Nuclear Reactions

- Types of nuclear reaction
  - Elastic scattering
  - Non-Elastic scattering
    - Inelastic scattering
    - Rearrangement reactions (transfer, knock out, fusion, fission …)

- ← we focus on elastic scattering here.

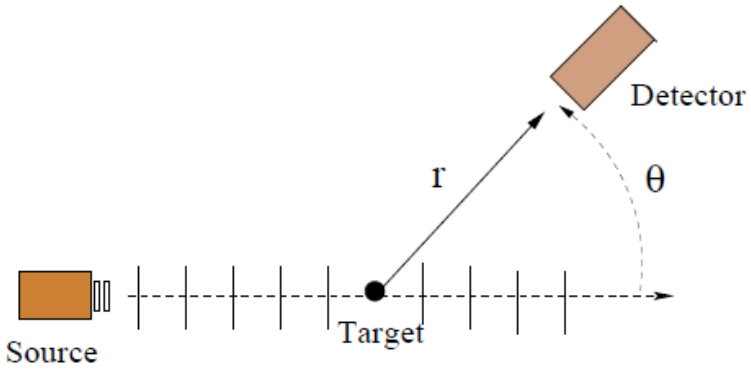


1. $^{11}\mathrm{Be} + {}^{208}\mathrm{Pb} \rightarrow {}^{11}\mathrm{Be} + {}^{208}\mathrm{Pb}$
2. $\rightarrow {}^{11}\mathrm{Be}^*(1/2^-) + {}^{208}\mathrm{Pb}$
3. $\rightarrow {}^{10}\mathrm{Be} + \mathrm{n} + {}^{208}\mathrm{Pb}$
4. $\rightarrow {}^{10}\mathrm{Be} + {}^{209}\mathrm{Pb}$

Introduction

- How to relate it with quantum theory?
- → The distribution of the products will be determined from the wave function
- → Solve Schrödinger equation

$$H\psi(\mathbf{r}) \equiv \left[-\frac{\hbar^2}{2m}\nabla^2 + V(r)\right]\psi(\mathbf{r}) = E\psi(\mathbf{r}).$$

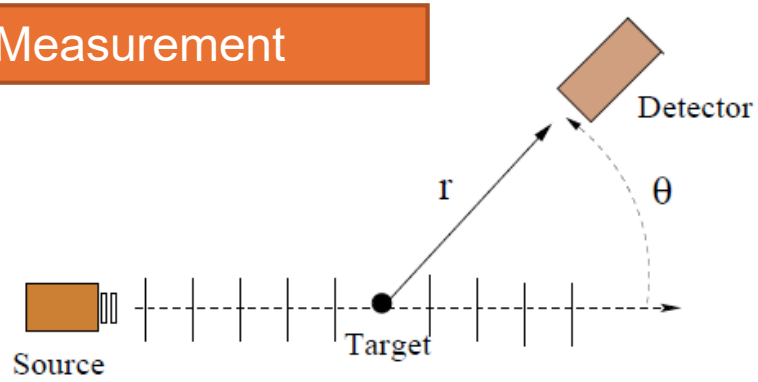


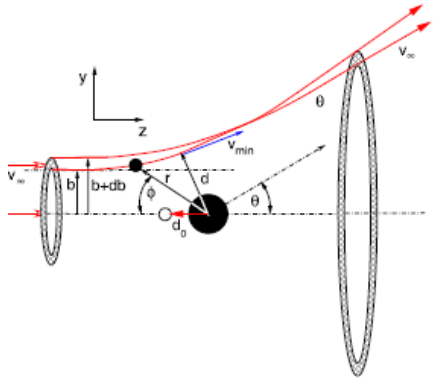
Compare with experiments
→ What kind of interaction and states(structure) describes the experiments best?

Cross section

Reaction Theory and Experiments can be compared with Cross Section

Measurement



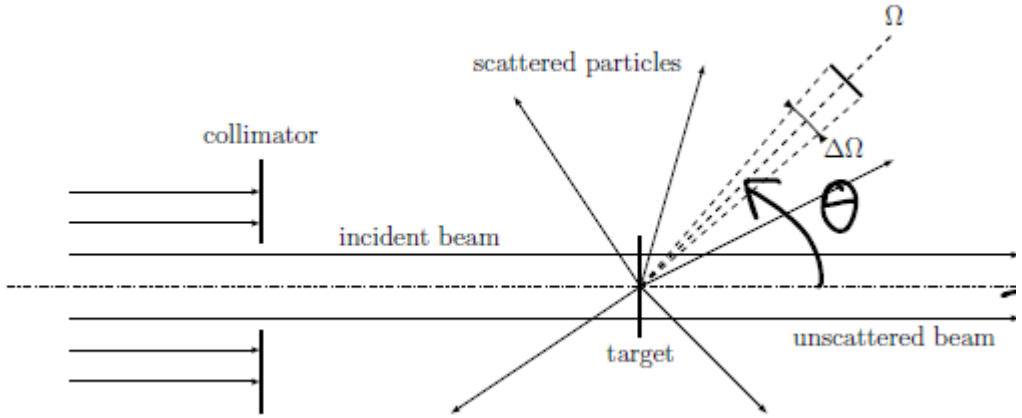


(Number of particles detected in solid angle/sec)
= (Number of incident particles/sec)*(effective area)/(incident area)
= (Number of incident particles/sec)
(effective area per target)(number of target particle in area)/(incident area)
= (Number of incident particles/sec)
(effective area per target)(target particle number density)*(target thickness)

(Cross section) = (effective area per reaction)
does not depends on the experimental condition (target thickness, beam size…)

Cross section

Reaction Theory and Experiments can be compared with Cross Section



incident flux F_i on N particles in the target

the number Δn of events detected in direction $\Omega = (\theta, \varphi)$
per unit time within the solid angle $\Delta\Omega$ is proportional to F_i and N :

$$\Delta n = F_i \, N \, \Delta\sigma$$

$[\Delta\sigma]$ = surface ; unit : barn $1 \text{ b} = 10^{-24}\text{cm}^2$

$$\frac{d\sigma}{d\Omega} = \lim_{\Delta\Omega \to 0} \frac{\Delta\sigma}{\Delta\Omega} = \frac{\Delta n}{F_i N \Delta\Omega}$$

Differential cross section

Schrödinger equation for central potential

- Schrödinger equation of one-body in spherical symmetric potential

$$H\psi(\mathbf{r}) \equiv \left[-\frac{\hbar^2}{2m}\nabla^2 + V(r)\right]\psi(\mathbf{r}) = E\psi(\mathbf{r}).$$

- Two-body problem ←→ One-body problem

$$\mathcal{H} = T_a + T_b + V(\boldsymbol{r}),$$

$$T_a = \frac{p_a^2}{2m_a} = -\frac{\hbar^2\Delta_{r_a}}{2m_a}$$

$$T_b = \frac{p_b^2}{2m_b} = -\frac{\hbar^2\Delta_{r_b}}{2m_b}$$

$$\boldsymbol{r} = \boldsymbol{r}_a - \boldsymbol{r}_b$$

$$\Psi_{\text{tot}}(\boldsymbol{r}_a, \boldsymbol{r}_b) = \Psi_{cm}(\boldsymbol{R}_{cm})\,\Psi(\boldsymbol{r})$$

$$\mathcal{H} = T_{\text{cm}} + T_r + V(\boldsymbol{r}),$$

$$T_{\text{cm}} = \frac{P_{\text{cm}}^2}{2M} = -\frac{\hbar^2\Delta_{R_{\text{cm}}}}{2M} \qquad M = m_a + m_b,$$

$$T_r = \frac{p^2}{2\mu} = -\frac{\hbar^2\Delta_r}{2\mu}, \qquad \mu = m_a m_b/M$$

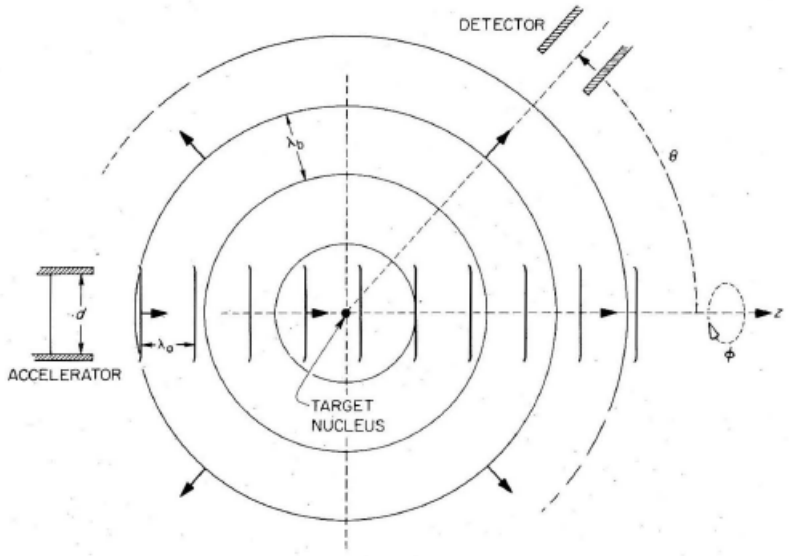
$$\boldsymbol{R}_{\text{cm}} = (m_a\boldsymbol{r}_a + m_b\boldsymbol{r}_b)/M$$

$$\boldsymbol{r} = \boldsymbol{r}_a - \boldsymbol{r}_b$$

Schrödinger equation for central potential

Stationary scattering States

$$H\,\Psi_{\boldsymbol{k}} = E\,\Psi_{\boldsymbol{k}} \qquad E = \hbar^2 k^2 / 2\mu$$



If interaction have finite range,
Asymptotic boundary condition

$$\psi^{asym} = A[e^{ikz} + f(\theta,\phi)\frac{e^{ik_f r}}{r}]$$

Incident wave + scattered wave

f : Scattering amplitude
f=0 , if there is no potential

Schrödinger equation for central potential

From continuity eq. and Schrodinger eq.

$$\frac{\partial \rho(\boldsymbol{x},t)}{\partial t} + \nabla \cdot \boldsymbol{j}(\boldsymbol{x},t) = 0 \qquad i\hbar\frac{\partial \psi}{\partial t} = [\hat{T} + \hat{V}]\psi$$

$$\rho(\boldsymbol{x},t) = |\psi(\boldsymbol{x},t)|^2$$

$$\frac{\partial \psi^*\psi}{\partial t} = \frac{\hbar}{2i\mu}\nabla \cdot [(\nabla\psi)^*\psi - \psi^*\nabla\psi] + \frac{1}{i\hbar}[\psi^*\hat{V}\psi - (\hat{V}\psi)^*\psi]$$

$$\frac{\partial \rho(\boldsymbol{x},t)}{\partial t} = -\nabla \cdot \boldsymbol{j}_{free} - \frac{i}{\hbar}\langle\psi| \left[\delta(\boldsymbol{r}-\boldsymbol{r}_i)\hat{V} - \hat{V}^\dagger\delta(\boldsymbol{r}-\boldsymbol{r}_i)\right] |\psi\rangle$$

The last term vanishes for local Hermitian interaction.
In general, interactions are non-local.

Free Current density

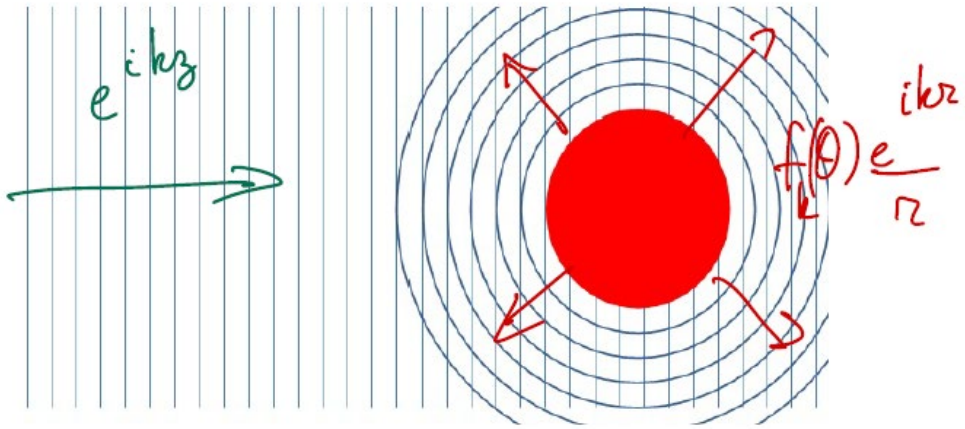
$$\boldsymbol{j}_{free} = \frac{\hbar}{2i\mu}(\psi^*\nabla\psi - \psi\nabla\psi^*) = \langle\psi|\frac{\hbar}{2i\mu}\left[\delta(\boldsymbol{r}-\boldsymbol{r}_i)\nabla_i - \nabla_i\delta(\boldsymbol{r}-\boldsymbol{r}_i)\right]|\psi\rangle$$

Plane wave Case : (density)*(velocity)

Schrödinger equation for central potential

$$\psi^{asym} = A[e^{ikz} + f(\theta,\phi)\frac{e^{ik_f r}}{r}]$$

$$\boldsymbol{j}_{free} = \frac{\hbar}{2i\mu}(\psi^*\nabla\psi - \psi\nabla\psi^*)$$



Then incident flux and scattered flux at large distance

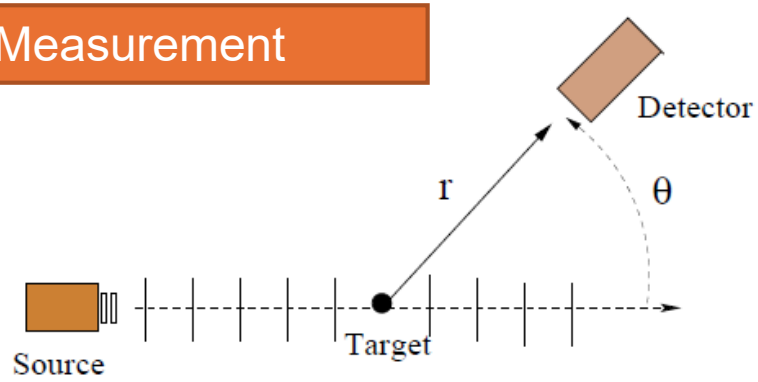
$$|\vec{j}_{inc}|d\sigma = \frac{k}{\mu}d\sigma$$

$$\vec{j}_{scatt}\cdot\hat{r} = \frac{k}{\mu}\frac{|f(\theta)|^2}{r^2}$$

~ number of particles scattered in particular direction
~ differential cross section

Schrödinger equation for central potential

Measurement



$$F_i = C\ j_i \qquad F_s(\Omega) = C\ j_s(\Omega)$$

$$dn = F_s(\Omega)\, dS \quad = \quad C\ j_s(\Omega)\ r^2 d\Omega$$

$$\begin{aligned} \frac{d\sigma}{d\Omega} &= \frac{dn}{F_i\, d\Omega} \\ &= \frac{r^2 j_s(\Omega)}{j_i} \\ &= |f_k(\theta)|^2 \end{aligned}$$

$$\vec{j}_{scatt} \cdot \hat{r} \quad = \quad \frac{k}{\mu} \frac{|f(\theta)|^2}{r^2}$$

$$d\sigma = \frac{(\#\text{ of particles scattered})/\text{sec}}{(\#\text{ of particles incident})/\text{area}/\text{sec}} = \frac{|j_{scatt}| r^2 d\Omega}{|j_{inc}|}$$

Theoretical cross section ← Scattering amplitude
(contains all information about V)

Partial Wave expansion

- Central potential→ Partial Wave expansion of wave function

$$\nabla^2 = \frac{1}{r^2}\frac{\partial}{\partial r}\left(r^2\frac{\partial}{\partial r}\right) + \frac{1}{r^2\sin\theta}\frac{\partial}{\partial\theta}\left(\sin\theta\frac{\partial}{\partial\theta}\right) + \frac{1}{r^2\sin^2\theta}\frac{\partial^2}{\partial\phi^2}$$

$$\begin{aligned}[H, L^2] &= 0 \\ [H, L_Z] &= 0\end{aligned}$$

$$L^2 = -\hbar^2\left[\frac{1}{\sin\theta}\frac{\partial}{\partial\theta}\left(\sin\theta\frac{\partial}{\partial\theta}\right) + \frac{1}{\sin^2\theta}\frac{\partial^2}{\partial\phi^2}\right].$$

$$H = -\frac{\hbar^2}{2m}\frac{1}{r^2}\frac{\partial}{\partial r}\left(r^2\frac{\partial}{\partial r}\right) + \frac{L^2}{2mr^2} + V(r).$$

$$L_z = -i\hbar\frac{\partial}{\partial\phi}$$

Spherical Harmonics

$$L_z Y_{\ell m}(\theta,\phi) = m\hbar Y_{\ell m}(\theta,\phi), \qquad L^2 Y_{\ell m}(\theta,\phi) = \ell(\ell+1)\hbar^2 Y_{\ell m}(\theta,\phi)$$

Schrödinger equation for central potential

$$-\frac{\hbar^2}{2m}\frac{1}{r^2}\frac{\partial}{\partial r}\left(r^2\frac{\partial R_{n\ell}}{\partial r}\right)+\left[V(r)+\frac{\hbar^2\ell(\ell+1)}{2mr^2}\right]R_{n\ell}(r)=E_{n\ell}R_{n\ell}(r).$$

$$\Psi(\boldsymbol{r})=\frac{1}{r}\sum_{l,m}u_{kl}(r)\,Y_l^m(\Omega),$$

$$\left(\frac{d^2}{dr^2}-\frac{l(l+1)}{r^2}-\frac{2\mu}{\hbar^2}V(r)+k^2\right)u_{kl}(r)=0 \qquad \frac{\hbar^2k^2}{2\mu}=E$$

Radial equation

← We changed 3-d equation into 1-d equation

- Full solution is a sum of different L solutions
- Each L can be solved separately as 1-d equation. ← Easy !

Schrödinger equation for central potential

- The same S.E.

$$H\psi(\mathbf{r}) \equiv \left[-\frac{\hbar^2}{2m}\nabla^2 + V(r)\right]\psi(\mathbf{r}) = E\psi(\mathbf{r}).$$

- Bound State Solution (E< 0)
  - Discrete Energies : Not all E <0 gives solution.
  - Wave function can be normalized : exponential decay of w.f. at large distance

- Scattering (unbound) State Solution (E > 0)
  - Continuous energies : All E > 0 gives solutions.
  - Wave function is not normalized : no decay of w.f. at large distance

- Same differential equation, but different boundary condition !

Free wave (Spherical Bessel Function)

No interaction : V = 0

$$\left(\frac{d^2}{dr^2} - \frac{l(l+1)}{r^2} + k^2\right) u_{kl}(r) = 0$$

$$u_{kl}(r) = x\, f_l(x), \text{ with } x = kr,$$

$$\left(\frac{d^2}{dx^2} + \frac{2}{x}\frac{d}{dx} + 1 - \frac{l(l+1)}{x^2}\right) f_l(x) = 0$$

Spherical Bessel equation
→ Solutions are
→ spherical Bessel (Neuman) functions.

$$j_0(x) = \frac{\sin x}{x} \qquad n_0(x) = \frac{\cos x}{x}$$

$$j_1(x) = \frac{\sin x}{x^2} - \frac{\cos x}{x} \qquad n_1(x) = \frac{\cos x}{x^2} + \frac{\sin x}{x}$$

$$j_l(x) \xrightarrow[x\to 0]{} \frac{x^l}{(2l+1)!!} \qquad n_l(x) \xrightarrow[x\to 0]{} \frac{(2l-1)!!}{x^{l+1}}$$

$$j_l(x) \xrightarrow[x\to\infty]{} \frac{1}{x}\sin(x - l\frac{\pi}{2}) \qquad n_l(x) \xrightarrow[x\to\infty]{} \frac{1}{x}\cos(x - l\frac{\pi}{2})$$

Any combination of
Spherical Bessel function
And Neuman function is
a solution of the equation

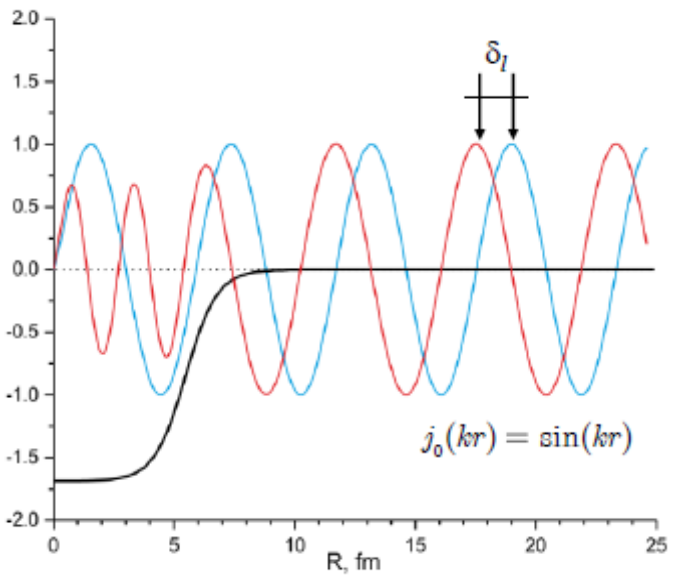
→ Boundary condition
determines the specific solution.

Phase Shift

- Nuclear interaction is short-range (finite range)
  → Any solution with interaction behaves as like a free solution at large distance.
  → just shifted in phase.

$$\left[-\frac{\hbar^2}{2\mu}\left(\frac{d^2}{dR^2}-\frac{L(L+1)}{R^2}\right)+V(R)-E\right]\chi_L(R)=0.$$

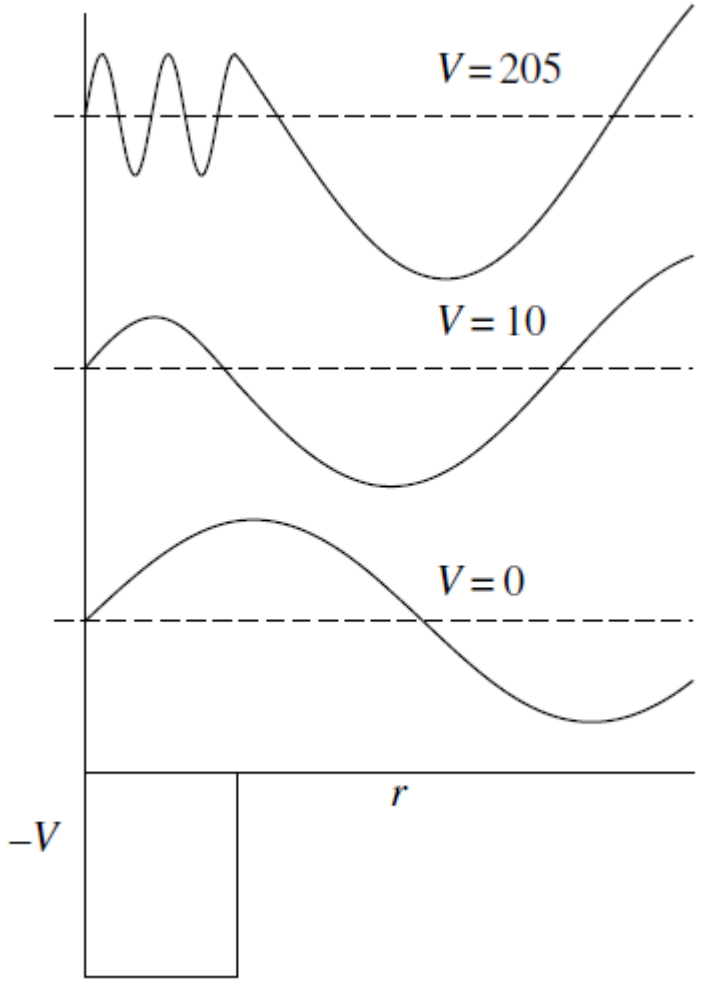
$$\left(\frac{d^2}{dr^2}-\frac{l(l+1)}{r^2}+k^2\right)u_{kl}^{\mathrm{as}}(r)=0$$



$$\begin{aligned} u_{kl}^{\mathrm{as}}(r) &= a_l\, kr\; j_l(kr) + b_l\, kr\; n_l(kr) \\ &\xrightarrow[r\to\infty]{} a_l\; \sin(kr - l\pi/2) + b_l\; \cos(kr - l\pi/2) \end{aligned}$$

$$u_{kl}^{\mathrm{as}}(r) \xrightarrow[r\to\infty]{} c_l\; \sin(kr - l\pi/2 + \delta_l)$$

Schrödinger equation for central potential



The asymptotics of the distorted wave

$$u_{kl}^{\mathrm{as}}(r) \xrightarrow[r\to\infty]{} c_l \ \sin(kr - l\pi/2 + \delta_l)$$

is phase shifted compared to the plane wave

$$a_l \, kr \ j_{kl}(r) \xrightarrow[r\to\infty]{} a_l \ \sin(kr - l\pi/2)$$

- $V(r) = 0 \quad \forall r \ \Rightarrow \ \delta_l = 0$: no shift
  the colliding particles don't see each other
- $V(r) < 0 \quad \forall r \ \Rightarrow \ \delta_l > 0$: wave attracted by the field
  the colliding particles tend to stick together
- $V(r) > 0 \quad \forall r \ \Rightarrow \ \delta_l < 0$: wave repulsed by the field
  the colliding particles repulse each other

Phase shift contains all information of the wave function at large distance.

How it is related to scattering amplitude?

Schrödinger equation for central potential

$$\psi^{asym} = A[e^{ikz} + f(\theta,\phi)\frac{e^{ik_f r}}{r}].$$

$$\sigma(\theta,\phi) = \frac{v_f}{v_i}|f(\theta,\phi)|^2.$$

$$e^{i\mathbf{k}\cdot\mathbf{r}} = \sum_{l=0}^{\infty}(2l+1)i^l j_l(kr)P_l(\cos\theta).$$

$$\begin{aligned} u_{kl}^{as}(r) &= a_l\, kr\; j_l(kr) + b_l\, kr\; n_l(kr) \\ &\xrightarrow[r\to\infty]{} a_l\; \sin(kr - l\pi/2) + b_l\; \cos(kr - l\pi/2) \end{aligned}$$

$$u_l(r) \approx \sin\left(kr - \frac{l\pi}{2} + \delta_l\right).$$

$$\begin{aligned} \psi_k(\boldsymbol{r}) &= \sum_l (2l+1)i^l \psi_l(r)P_l(\cos\theta), \\ &\to \sum_l (2l+1)i^l \left(\frac{\sin(\rho - \frac{l}{2}\pi)}{\rho} + i^{-l}\frac{e^{2i\delta_l}-1}{2i}\frac{e^{i\rho}}{\rho}\right)P_L(\cos\theta) \\ &\simeq e^{ikz} + \left(\sum_l (2l+1)\frac{e^{2i\delta_l}-1}{2ik}P_l(\cos\theta)\right)\frac{e^{ikr}}{r}, \quad \rho \gg 1 \end{aligned}$$

$$S_l = e^{2i\delta_l}$$ S-matrix

$$\boxed{f(\theta) = \sum_l (2l+1)\frac{S_l - 1}{2ik}P_l(\cos\theta), \quad \frac{d\sigma}{d\Omega} = |f(\theta)|^2}$$

Schrödinger equation for central potential

$$\boxed{f(\theta) = \sum_l (2l+1)\frac{S_l - 1}{2ik} P_l(\cos\theta), \qquad \frac{d\sigma}{d\Omega} = |f(\theta)|^2}$$

$$\frac{\mathrm{d}\sigma}{\mathrm{d}\Omega} = \frac{1}{k^2}\left|\sum_{l=0}^{\infty}(2l+1)\mathrm{e}^{\mathrm{i}\delta_l}\sin(\delta_l)P_l(\cos\theta)\right|^2$$

$$\sigma_{\mathrm{tot}} = 2\pi\int \mathrm{d}\theta\,\sin\theta\,\frac{\mathrm{d}\sigma}{\mathrm{d}\Omega}(\theta) = \frac{4\pi}{k^2}\sum_{l=0}^{\infty}(2l+1)\sin^2\delta_l.$$

Total (elastic) cross section

$$\left[-\frac{\hbar^2}{2\mu}\left(\frac{d^2}{dR^2} - \frac{L(L+1)}{R^2}\right) + V(R) - E\right]\chi_L(R) = 0.$$

Centrifugal barrier $\frac{l(l+1)}{r^2}$
ensures $\delta_l \xrightarrow[l\to\infty]{} 0$
$\Rightarrow$ limited sum

Differential equation

$$\psi_k(\boldsymbol{r}) = \sum_L i^L (2L+1) P_L(\cos\theta) \frac{\chi_L(r;k)}{kr}$$

$$\left[-\frac{\hbar^2}{2\mu}\left(\frac{d^2}{dR^2} - \frac{L(L+1)}{R^2}\right) + V(R) - E\right]\chi_L(R) = 0.$$

Numerical solution from Numerov, or RK4 methods.

Initial Value: one can use arbitrary values for scattering problem.

$$u_L(0) = 0 \text{ and } u'_L(0) \neq 0$$

$$\chi_L(R) = B_L u_L(R)$$

Internal solution r < a.

We want to determine phase shift from the wave function solution.

$$R^{\text{as}}_{kl}(r) = c_l\left[\cos\delta_l j_l(kr) + \sin\delta_l n_l(kr)\right]$$

Differential equation

Because of r, it can be convenient to define Ricatti-Bessel, Ricatti-Hankel functions

$$\left[\frac{\hbar^2}{2\mu}\frac{d^2}{dr^2} - \frac{\hbar^2}{2\mu}\frac{l(l+1)}{r^2} - V(r) + E\right] u_{l,k}(r) = 0.$$

$$\begin{aligned} u_{l,k}(r \gg 1) &\propto \sin(x - l\frac{\pi}{2} + \delta_l), \quad x = kr, \\ &\propto \cos(\delta_l)\sin(x - l\frac{\pi}{2}) + \sin(\delta_l)\cos(x - l\frac{\pi}{2}) \end{aligned}$$

$$\begin{aligned} \hat{j}_l(x) &\equiv x j_l(x) \to_{x\gg 1} \sin(x - l\frac{\pi}{2}), \\ \hat{n}_l(x) &\equiv x n_l(x) \to_{x\gg 1} \cos(x - l\frac{\pi}{2}) \end{aligned}$$

$$\boxed{\hat{y}_l(x) = x y_l(x) = -x n_l(x)}$$

Scipy convention

$$\hat{h}_l^{(\pm)}(x) \propto \hat{n}_l(x) \pm i\hat{j}_l(x) \to_{x\gg 1} e^{\pm i(x - l\frac{\pi}{2})}$$

Differential equation

$$\left[\frac{\hbar^2}{2\mu}\frac{d^2}{dr^2} - \frac{\hbar^2}{2\mu}\frac{l(l+1)}{r^2} - V(r) + E\right] u_{l,k}(r) = 0.$$

$$\begin{aligned} u_{l,k}(r \gg 1) &\propto \sin(x - l\frac{\pi}{2} + \delta_l), \quad x = kr, \\ &\propto \cos(\delta_l)\sin(x - l\frac{\pi}{2}) + \sin(\delta_l)\cos(x - l\frac{\pi}{2}), \\ &\propto \cos(\delta_l)\left[\hat{j}_l(x) + \tan(\delta_l)\hat{n}_l(x)\right] \end{aligned}$$

$$u_{l,k}(r \gg 1) \propto \frac{i}{2}e^{-i\delta_l}\left[\hat{h}_L^{(-)}(x) - e^{2i\delta_l}\hat{h}_l^{(+)}(x)\right]$$

S-matrix is unknown yet.
We want to get S-matrix from Boundary matching condition.

Differential equation

$$
\begin{aligned}
u_{l,k}(r \gg 1) &\propto \sin(x - l\frac{\pi}{2} + \delta_l), \quad x = kr, \\
&\propto \cos(\delta_l)\sin(x - l\frac{\pi}{2}) + \sin(\delta_l)\cos(x - l\frac{\pi}{2}), \\
&\propto \cos(\delta_l)\left[\hat{j}_l(x) + \tan(\delta_l)\hat{n}_l(x)\right]
\end{aligned}
$$

$$
\Rightarrow \quad u(r_m) = u_{ext}(r_m), \quad \frac{du(r_m)}{dr} = \frac{du_{ext}(r_m)}{dr}
$$

R-matrix

$$
R_l \equiv \frac{1}{a}\frac{u_{l,k}(a)}{du_{l,k}(a)/dr}
$$

$$
\begin{aligned}
\gamma_l &\equiv \frac{du_l(a)/dr}{u_l(a)} = \frac{1}{aR_l}, \\
&= \frac{k\frac{d}{d\rho}\left[\hat{j}_l(\rho) - \tan\delta_l\hat{y}_l(\rho)\right]}{\left[\hat{j}_l(\rho) - \tan\delta_l\hat{y}_l(\rho)\right]}, \quad \rho = ka
\end{aligned}
$$

Differential equation

$$\tan\delta_l = \frac{\gamma_l \hat{j}_l(\rho) - k\frac{d\hat{j}_l(\rho)}{d\rho}}{\gamma_l \hat{y}_l(\rho) - k\frac{d\hat{y}_l(\rho)}{d\rho}} = \frac{\hat{j}_l(\rho) - \rho R_l \frac{d\hat{j}_l(\rho)}{d\rho}}{\hat{y}_l(\rho) - \rho R_l \frac{d\hat{y}_l(\rho)}{d\rho}}$$

We can get gamma_l from numerical solution.

$$\gamma_l \equiv \frac{du_l(a)/dr}{u_l(a)} = \frac{1}{aR_l},$$

Scattering problem

1. solving radial Schrodinger equation for each L
   → numerical integration methods (Numerov, RK4 …)

2. Match boundary condition to determine phase shift for L

3. Compute cross section by summing each L contributions

Here, let us use RK4 method for ordinary differential equation.

Code

- Here we do not focus on the numerical algorithm, but develop a code to solve the scattering problem.
- Let us use python and scipy package.
- Spherical Bessel Function is available in scipy.special
- ODE solver is available in scipy.integrate

- Keep in mind of units : Let us use natural units and everything in fermi (and its powers).

```
import numpy as np
from scipy.special import (spherical_jn, spherical_yn,
                           riccati_jn, riccati_yn,
                           eval_legendre)
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt

# — Physical constants ————————————————————————————————————————
hbarc    = 197.3269788   # MeV·fm
alpha_em = 1.0/137.035999
amu      = 931.4940954   # MeV/c²
mpi      = 138.03898     # MeV/c²
mass_N   = 938.9187      # MeV/c²
```

Code


```python
# — Potential models ————————————————————————————————————————
def sq_well(r, V0=1.0, R0=2.0):
    """Square-well potential  V(r) = -V0  for r ≤ R0,  else 0."""
    r = np.asarray(r, dtype=float)
    return np.where(r <= R0, -V0, 0.0)


def WSPOT(r, V0=1.0, R0=2.0, a0=0.65):
    """Woods-Saxon potential."""
    return -V0 / (1.0 + np.exp((r - R0) / a0))
```

```python
# Normalised regular BC: u(r0) = 1,  u'(r0) = (L+1)/r0
y0 = np.array([1.0, (L + 1) / r0])

def derivs(r, y):
    u, du = y
    ddu = (L*(L+1)/r**2 + 2.0*mu*pot_func(r) - 2.0*mu*E) * u
    return [du, ddu]
```

$$\left(\frac{d^2}{dr^2} - \frac{l(l+1)}{r^2} - \frac{2\mu}{\hbar^2}V(r) + \frac{2\mu E}{\hbar^2}\right) u_l = 0.$$

$$\begin{aligned} y_0(r) &= u_l(r) \quad y_1(r) = u_l'(r), \\ \frac{d}{dr}y_0 &= y_1, \\ \frac{d}{dr}y_1 &= \left(\frac{l(l+1)}{r^2} + \frac{2\mu}{\hbar^2}V(r) - \frac{2\mu E}{\hbar^2}\right) y_0. \end{aligned}$$

Code

scipy.integrate.

solve_ivp

```
solve_ivp(fun, t_span, y0, method='RK45', t_eval=None, dense_output=False,
events=None, vectorized=False, args=None, **options)
```
[source]

Solve an initial value problem for a system of ODEs.

This function numerically integrates a system of ordinary differential equations given an initial value:

```
dy / dt = f(t, y)
y(t0) = y0
```

Code

fun : *callable*

Right-hand side of the system: the time derivative of the state `y` at time `t`. The calling signature is `fun(t, y)`, where `t` is a scalar and `y` is an ndarray with `len(y) = len(y0)`. Additional arguments need to be passed if `args` is used (see documentation of `args` argument). `fun` must return an array of the same shape as `y`. See *vectorized* for more information.

t_span : *2-member sequence*

Interval of integration (t0, tf). The solver starts with t=t0 and integrates until it reaches t=tf. Both t0 and tf must be floats or values interpretable by the float conversion function.

y0 : *array_like, shape (n,)*

Initial state. For problems in the complex domain, pass *y0* with a complex data type (even if the initial value is purely real).

Code

method : ***string or*** `OdeSolver`***, optional***

Integration method to use:

- 'RK45' (default): Explicit Runge-Kutta method of order 5(4) [1]. The error is controlled assuming accuracy of the fourth-order method, but steps are taken using the fifth-order accurate formula (local extrapolation is done). A quartic interpolation polynomial is used for the dense output [2]. Can be applied in the complex domain.
- 'RK23': Explicit Runge-Kutta method of order 3(2) [3]. The error is controlled assuming accuracy of the second-order method, but steps are taken using the third-order accurate formula (local extrapolation is done). A cubic Hermite polynomial is used for the dense output. Can be applied in the complex domain.
- 'DOP853': Explicit Runge-Kutta method of order 8 [13]. Python implementation of the "DOP853" algorithm originally written in Fortran [14]. A 7-th order interpolation polynomial accurate to 7-th order is used for the dense output. Can be applied in the complex domain.
- 'Radau': Implicit Runge-Kutta method of the Radau IIA family of order 5 [4]. The error is controlled with a third-order accurate embedded formula. A cubic polynomial which satisfies the collocation conditions is used for the dense output.
- 'BDF': Implicit multi-step variable-order (1 to 5) method based on a backward differentiation formula for the derivative approximation [5]. The implementation follows the one described in [6]. A quasi-constant step scheme is used and accuracy is enhanced using the NDF modification. Can be applied in the complex domain.
- 'LSODA': Adams/BDF method with automatic stiffness detection and switching [7], [8]. This is a wrapper of the Fortran solver from ODEPACK.

```
>>> import numpy as np
>>> from scipy.integrate import solve_ivp
>>> def exponential_decay(t, y): return -0.5 * y
>>> sol = solve_ivp(exponential_decay, [0, 10], [2, 4, 8])
>>> print(sol.t)
[ 0.          0.11487653  1.26364188  3.06061781  4.81611105  6.57445806
  8.33328988 10.        ]
>>> print(sol.y)
[[2.         1.88836035 1.06327177 0.43319312 0.18017253 0.07483045
  0.03107158 0.01350781]
 [4.         3.7767207  2.12654355 0.86638624 0.36034507 0.14966091
  0.06214316 0.02701561]
 [8.         7.5534414  4.25308709 1.73277247 0.72069014 0.29932181
  0.12428631 0.05403123]]
```

Code


```
k = np.sqrt(2.0 * mu * E)

r_max = r_match + 2.0

#   Since the Schrödinger equation is linear, the overall scale of u
#   is arbitrary; only the log-derivative matters for the phase shift.
#   We therefore normalise the IC to O(1) by dividing by r0^(L+1):
#      u(r0) / r0^(L+1) = 1,   u'(r0) / r0^(L+1) = (L+1)/r0
r0 = 0.01
rr = np.linspace(r0, r_max, npts)

# Normalised regular BC: u(r0) = 1,  u'(r0) = (L+1)/r0
y0 = np.array([1.0, (L + 1) / r0])

def derivs(r, y):
    u, du = y
    ddu = (L*(L+1)/r**2 + 2.0*mu*pot_func(r) - 2.0*mu*E) * u
    return [du, ddu]

# Use DOP853 (8th-order) with tight tolerances for high accuracy,
# especially important for large L where phase shifts are tiny.
usol = solve_ivp(derivs, [rr[0], rr[-1]], y0, t_eval=rr,
                 method='DOP853', rtol=1e-13, atol=1e-15)
```

$$\left(\frac{d^2}{dr^2} - \frac{l(l+1)}{r^2} - \frac{2\mu}{\hbar^2}V(r) + 2\mu E\right) u_l = 0.$$

Initial condition

$$u_l(r \to 0) = c\, r^{l+1}$$

Scattering problem: one can choose arbitrary values for initial condition.

Bound state problem: one have to search correct initial condition which gives bound state so lution. (shooting method.)

Code


```
# --- R-matrix matching ---
idx     = (np.abs(rr - r_match)).argmin()
u_m     = usol.y[0][idx]
dudr_m  = usol.y[1][idx]
r_m     = rr[idx]
rho_m   = k * r_m

# R = u / (r · du/dr)
Rmat = u_m / (r_m * dudr_m)

jn_s, djn_s = riccati_jn(L, rho_m)
yn_s, dyn_s = riccati_yn(L, rho_m)

rj  = jn_s[L];  drj = djn_s[L]
ry  = yn_s[L];  dry = dyn_s[L]

num = rj  - rho_m * Rmat * drj
den = ry  - rho_m * Rmat * dry
delta = np.arctan(num / den)

return rr, usol.y[0], usol.y[1], delta
```

Solve matching condition for S-matrix

$$R_l \equiv \frac{1}{a}\frac{u_{l,k}(a)}{du_{l,k}(a)/dr}$$

$$\gamma_l \equiv \frac{du_l(a)/dr}{u_l(a)} = \frac{1}{aR_l},$$

$$= \frac{k\frac{d}{d\rho}\left[\hat{j}_l(\rho) - \tan\delta_l \hat{y}_l(\rho)\right]}{\left[\hat{j}_l(\rho) - \tan\delta_l \hat{y}_l(\rho)\right]}, \quad \rho = ka$$

Code


```python
# ── Cross-section helpers ──────────────────────────────────────────
def total_cross_section(k, delta_array):
    """
    σ_tot = (4π/k²) Σ_L (2L+1) sin²(δ_L)

    Parameters
    ----------
    k           : float   - wave vector [fm^-1]
    delta_array : ndarray - phase shifts for L = 0, 1, ..., Lmax-1  [rad]

    Returns
    -------
    sigma_tot : float [fm²]
    """
    L_list = np.arange(len(delta_array))
    return 4.0*np.pi/k**2 * np.sum((2*L_list + 1) * np.sin(delta_array)**2)
```

$$\sigma_{\text{tot}} = 2\pi \int \mathrm{d}\theta \sin\theta \frac{\mathrm{d}\sigma}{\mathrm{d}\Omega}(\theta) = \frac{4\pi}{k^2} \sum_{l=0}^{\infty} (2l+1) \sin^2 \delta_l.$$

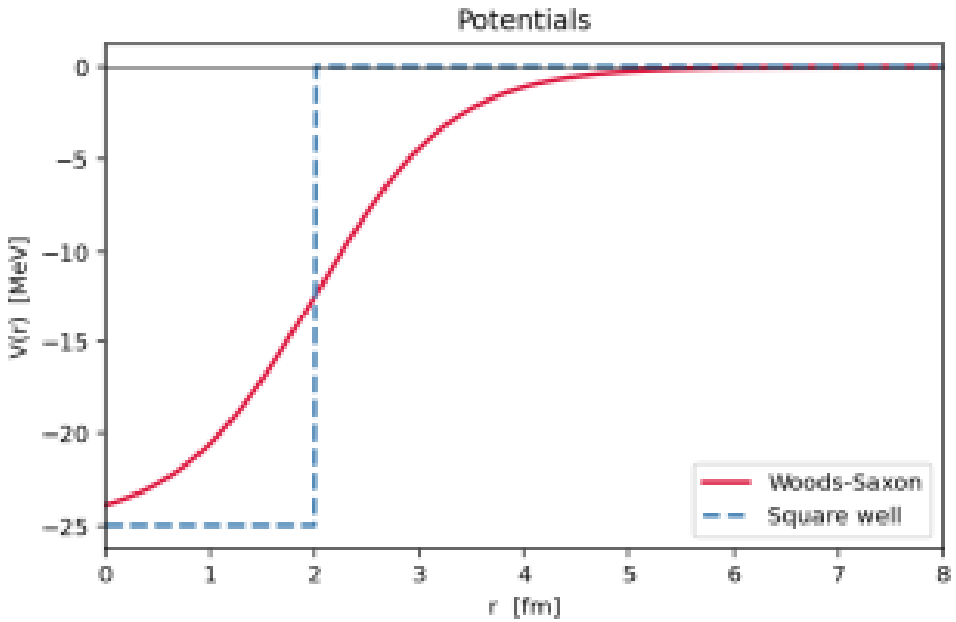
Code

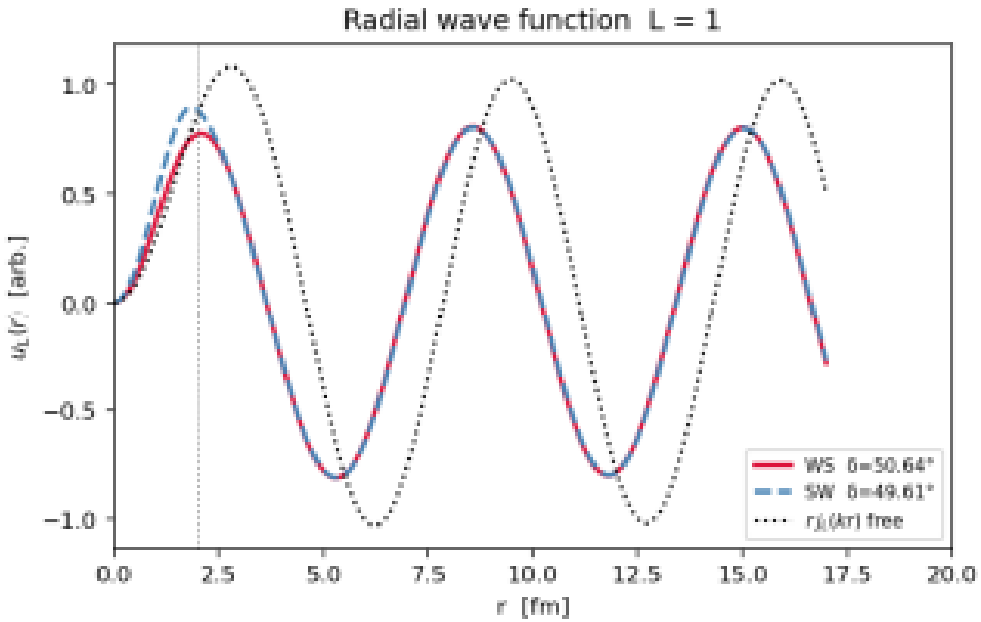

```python
def differential_cross_section(k, delta_array, thetas):
    """
    dσ/dΩ(ϑ) = |f(ϑ)|²

    f(ϑ) = (1/k) Σ_L (2L+1) sin(δ_L) exp(iδ_L) P_L(cosϑ)

    Parameters
    ----------
    k           : float   - wave vector [fm^-1]
    delta_array : ndarray - phase shifts [rad]
    thetas      : ndarray - scattering angles [rad]

    Returns
    -------
    dσ/dΩ : ndarray [fm²/sr]
    """
    ftheta = np.zeros(len(thetas), dtype=complex)
    cos_th = np.cos(thetas)
    for L, delta in enumerate(delta_array):
        ftheta += ((2*L + 1) * np.sin(delta) * np.exp(1j*delta)
                   * eval_legendre(L, cos_th))
    ftheta /= k
    return np.abs(ftheta)**2
```

$$f(\theta) = \sum_l (2l+1)\frac{S_l - 1}{2ik} P_l(\cos\theta), \qquad \frac{d\sigma}{d\Omega} = |f(\theta)|^2$$

$$\frac{\mathrm{d}\sigma}{\mathrm{d}\Omega} = \frac{1}{k^2}\left|\sum_{l=0}^{\infty}(2l+1)\mathrm{e}^{\mathrm{i}\delta_l}\sin(\delta_l)P_l(\cos\theta)\right|^2$$

Code


Potentials
V(r) [MeV]
r [fm]
Woods-Saxon
Square well


Radial wave function L = 1
r [fm]
WS δ=50.64°
SW δ=49.61°
rj_L(kr) free

Numerical Quantum Mechanics: Harmonic Oscillator

Young-Ho Song (IRIS, IBS)

IRIS 중이온가속기연구소
Institute for Rare Isotope Science

2026.03.31. Yonsei University

Basis expansion

$$\left(\frac{d^2}{dr^2} - \frac{l(l+1)}{r^2} - \frac{2\mu}{\hbar^2}V(r) + 2\mu E\right) u_l = 0.$$

- The same radial equation describes both bound state and scattering state.
- We solved Sch. Eq. as differential equation for each (L, E).

- Bound state : we search binding energy which satisfies boundary condition.
  → Shooting method.

- We can solve the same problem as matrix eigenvalue problem or variational method.

Basis expansion

$$H|\psi\rangle = E|\psi\rangle$$

Suppose the ortho-normal basis function exists,

$$|\psi\rangle = \sum_i c_i|\phi_i\rangle, \quad \langle\phi_i|\phi_j\rangle = \delta_{ij}$$

$$\langle\phi_i|H|\psi\rangle = \sum_j \langle\phi_i|H|\phi_j\rangle c_j = E\langle\phi_i|\psi\rangle = E\sum_j c_j\langle\phi_i|\phi_j\rangle = Ec_i$$

$$H_{ij} \equiv \langle\phi_i|H|\phi_j\rangle$$

$$\sum_j H_{ij}c_j = Ec_i \quad \Rightarrow \quad \boldsymbol{Hc} = E\boldsymbol{c}$$

Matrix eigenvalue problem

Variational theorem

Infinite number of basis function → Exact, not practical
Finite number of basis function → Approximation

$$|\Phi\rangle \simeq \sum_i^N c_i |\phi_i\rangle$$

Variational Theorem

$$\begin{aligned}
|\Phi\rangle &= \sum_n c_n |\psi_n\rangle, \quad H|\psi_n\rangle = E_n|\psi_n\rangle \\
\frac{\langle\Phi|H|\Phi\rangle}{\langle\Phi|\Phi\rangle} &= \frac{\sum_{nm} c_n^* c_m \langle\psi_n|H|\psi_m\rangle}{\sum_{nm} c_n^* c_m \langle\psi_n|\psi_m\rangle} = \frac{\sum_n |c_n|^2 E_n}{\sum_n |c_n|^2} \\
&\geq E_0
\end{aligned}$$

1d Coordinate basis

- In fact, coordinate space wave function can be Considered as a kind of basis expansion.

$$|\psi\rangle \simeq \sum_{i}^{N} \psi(r_i)|r_i\rangle, \quad r_i = i * h$$

$$\langle r_i|\hat{H}|r_j\rangle = \langle r_i| - \frac{\hbar^2}{2\mu}\frac{d^2}{dr^2}|r_j\rangle + \langle r_i|\frac{l(l+1)}{2\mu r^2} + V(r)|r_j\rangle$$

1d Coordinate basis

Matrix elements of kinetic term:
finite difference approximation

$$\frac{d^2}{dr^2} f(r_i) \simeq \frac{f(r_{i-1}) - 2f(r_i) + f(r_{i+1})}{h^2},$$

$$\langle r_i | \frac{d^2}{dr^2} | r_j \rangle \simeq \frac{1}{h^2}\delta_{i,j-1} - \frac{2}{h^2}\delta_{i,j} + \frac{1}{h^2}\delta_{i,j+1}$$

Matrix elements of potential term:
local potential case

$$\langle r_i | \frac{l(l+1)}{2\mu r^2} + V(r) | r_j \rangle = \left(\frac{l(l+1)}{2\mu r_i^2} + V(r_i) \right) \delta_{ij}$$

1d Coordinate basis

(With boundary condition $u_0 = u_{N+1} = 0.$) → Bound state problem case.
Scattering problem is more complicate.

$$\frac{-\hbar^2}{2\mu h^2}\begin{bmatrix} -2 & 1 & 0 & \cdots & 0 \\ 1 & -2 & 1 & \cdots & 0 \\ 0 & 1 & -2 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 1 \\ 0 & 0 & \cdots & 1 & -2 \end{bmatrix}_{ij} + \begin{bmatrix} \frac{l(l+1)}{2\mu r_1^2} + V(r_1) & 0 & 0 & \cdots & 0 \\ 0 & \frac{l(l+1)}{2\mu r_2^2} + V(r_2) & 0 & \cdots & 0 \\ 0 & 0 & \frac{l(l+1)}{2\mu r_3^2} + V(r_3) & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 \\ 0 & 0 & \cdots & 0 & \frac{l(l+1)}{2\mu r_N^2} + V(r_N) \end{bmatrix}_{ij}$$

$$H_{ii} \rightarrow \frac{\hbar^2}{2\mu h^2}2 - \frac{l(l+1)}{2\mu r_i^2} + V(r_i),$$

$$H_{i\neq j} \rightarrow -\frac{\hbar^2}{2\mu h^2}$$

$$\sum_{j=1}^{N} H_{ij} c_j^{(n)} = \lambda_n c_i^{(n)}$$

$$|\psi\rangle \simeq \sum_i^N \psi(r_i)|r_i\rangle, \quad r_i = i * h$$

1d Coordinate basis

How to solve matrix eigenvalue problem?

$$\sum_{j=1}^{N} H_{ij} c_j^{(n)} = \lambda_n c_i^{(n)}$$

There are many numerical algorithm to solve the matrix eigenvalue problem

We are not going into details of each alg orithm, but use scipy packages to solve the problem.

Dense symmetric / Hermitian matrices

- **QR algorithm** — implicitly shifted, deflation; all eigenvalues, $O(n^3)$
- **Divide & conquer** — secular equation, rank-1; all evals, fast in practice
- **MRRR** — multiple rel. robust repres.; $O(n^2)$ work, LAPACK default

↑ All three require prior reduction to tridiagonal form (Householder / Lanczos)

Dense non-symmetric matrices

- **Francis QR** — double-shift, bulge chasing; complex eigenvalues, $O(n^3)$
- **Hessenberg reduction** — Householder similarity; pre-step, $O(n^3)$ once
- **Schur decomposition** — $T = Q H Q^H$ (upper tri.); eigenvectors via back-sub

Sparse matrices — Krylov subspace methods

- **Lanczos** — symmetric only; extremal eigenvalues first; reorthog. needed in practice
- **Arnoldi iteration** — general (non-symmetric); Hessenberg projection; basis of ARPACK/MATLAB
- **IRAM / FEAST** — implicit restart (IRAM); contour integration (FEAST); interior eigenvalues

Power-type / projection methods

- **Power iteration** — largest $|\lambda|$ only; rate: $|\lambda_2/\lambda_1|$; simple, rarely used alone
- **Inverse iteration** — shift σ: $(A-\sigma I)^{-1}x$; targets λ nearest σ; eigenvector refinement
- **Rayleigh quotient iter.** — adaptive shift $\rho = x^T Ax / x^T x$; cubic convergence; one eigenvalue at a time

1d Coordinate basis

scipy.linalg.

eigh

eigh(*a, b=None, *, lower=True, eigvals_only=False, overwrite_a=False, overwrite_b=False, type=1, check_finite=True, subset_by_index=None, subset_by_value=None, driver=None*) [source]

Solve a standard or generalized eigenvalue problem for a complex Hermitian or real symmetric matrix.

Find eigenvalues array `w` and optionally eigenvectors array `v` of array `a`, where `b` is positive definite such that for every eigenvalue λ (i-th entry of w) and its eigenvector `vi` (i-th column of `v`) satisfies:

```
                a @ vi = λ * b @ vi
vi.conj().T @ a @ vi = λ
vi.conj().T @ b @ vi = 1
```

In the standard problem, `b` is assumed to be the identity matrix.

Parameters:

a : *(M, M) array_like*

A complex Hermitian or real symmetric matrix whose eigenvalues and eigenvectors will be computed.

b : *(M, M) array_like, optional*

A complex Hermitian or real symmetric definite positive matrix in. If omitted, identity matrix is assumed.

lower : *bool, optional*

Whether the pertinent array data is taken from the lower or upper triangle of `a` and, if applicable, `b`. (Default: lower)

eigvals_only : *bool, optional*

Whether to calculate only eigenvalues and no eigenvectors. (Default: both are calculated)

Returns:

w : *(N,) ndarray*

The N (N<=M) selected eigenvalues, in ascending order, each repeated according to its multiplicity.

v : *(M, N) ndarray*

The normalized eigenvector corresponding to the eigenvalue `w[i]` is the column `v[:,i]`. Only returned if `eigvals_only=False`.

H.O. wave function

Consider Harmonic Oscillator Hamiltonian.

For simplicity, let us use natural units and set m=1, w=1

$$H = -\frac{\hbar^2}{2m}\nabla^2 + \frac{1}{2}m\omega^2 r^2$$

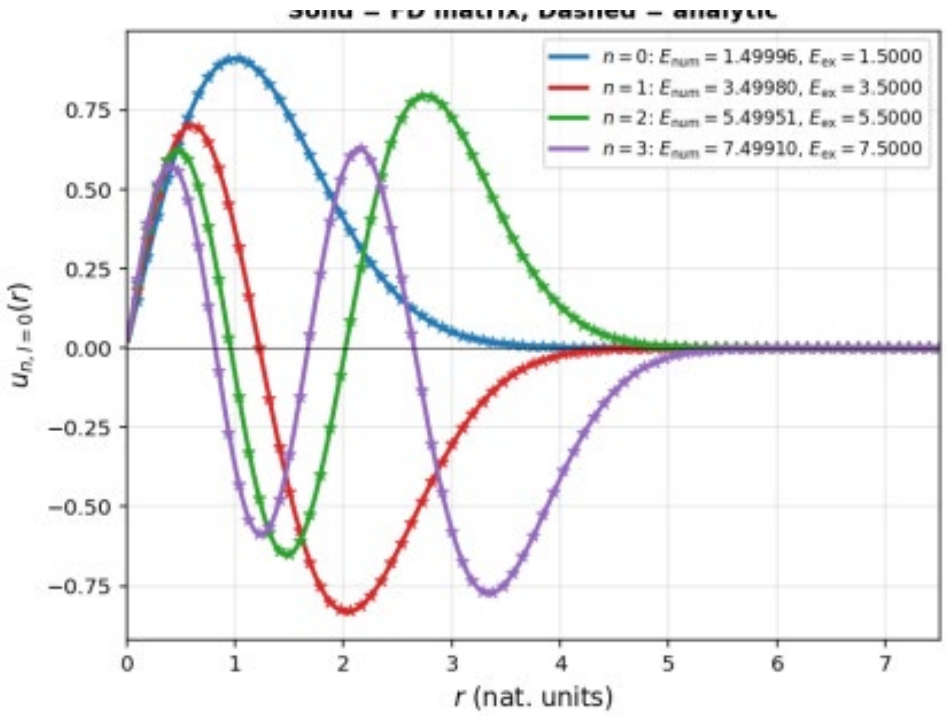
```
dr = r_max / (N_grid + 1)
r  = np.arange(1, N_grid + 1) * dr
```

```
H = build_hamiltonian_fd(r, V_harmonic_oscillator, l=l)
E_num, u_num = eigh(H, subset_by_index=[0, n_states - 1])
```

```
# Kinetic energy matrix: T = -1/2 * d^2/dr^2
#   T_{ii}     = +1/dr^2          (diagonal)
#   T_{i,i+1}  = T_{i,i-1} = -1/(2 dr^2)  (off-diagonal)
T_diag    =  np.ones(N)   / dr**2
T_offdiag = -np.ones(N-1) / (2.0 * dr**2)

T = (np.diag(T_diag)
     + np.diag(T_offdiag, k=+1)
     + np.diag(T_offdiag, k=-1))

# Potential: centrifugal barrier + external potential
V_centrifugal = l * (l + 1) / (2.0 * r**2)
V_ext         = V_func(r)
V = np.diag(V_centrifugal + V_ext)
```

H.O. wave function

```
======================================================================
 PART 1: HO Coordinate Space — Finite Difference (l=0, N=400, r_max=8)
======================================================================
   n     E_numerical     E_exact       |Error|    Rel.Err.
----------------------------------------------------------
   0      1.49993781      1.5000      6.22e-05    4.15e-05
   1      3.49968903      3.5000      3.11e-04    8.88e-05
   2      5.49924119      5.5000      7.59e-04    1.38e-04
   3      7.49859427      7.5000      1.41e-03    1.87e-04
   4      9.49774823      9.5000      2.25e-03    2.37e-04
   5     11.49670305     11.5000      3.30e-03    2.87e-04
```



We could get the correct results for (L=0, N=400).

However, coordinate basis expansion is not practical because the matrix size can be very large.

→ Better basis function which reduce the size of matrix
→ H.O. basis is commonly used.

HO basis

- The Harmonic Oscillator wave functions are used in many cases,

$$H = -\frac{\hbar^2}{2m}\nabla^2 + \frac{1}{2}m\omega^2 r^2 = \frac{\hbar^2}{2mb^2}\left(-b^2\nabla_r^2 + \frac{r^2}{b^2}\right) = \frac{\hbar\omega}{2}\left(-b^2\nabla_r^2 + \frac{r^2}{b^2}\right), \qquad \nu = \frac{m\omega}{2\hbar} = \frac{1}{2b^2}, \quad \frac{\hbar^2}{2mb^2} = \frac{\hbar\omega}{2}$$

- HO wave function

$$(-\frac{\hbar^2}{2m}\nabla^2 + U(r))\psi_{klm}(\boldsymbol{r}) = E_{kl}\psi_{klm}(\boldsymbol{r}), \qquad E_{kl} = \hbar\omega(N + \frac{3}{2}) = \hbar\omega(2k + l + \frac{3}{2}), \quad n \geq 0,\ k = (N-l)/2 \geq 0$$

1d H.O. ~ Hermite polynomial
3d H.O. ~ 3 Cartesian 1d H.O. or
~(Associate Laguerre polynomial)
*(Spherical Harmonics)

$$\begin{aligned}
\psi_{nlm}(\boldsymbol{r}) &= \frac{u_{nl}(r)}{r}Y_{lm}(\hat{\boldsymbol{r}}) \\
\frac{u_{kl}(r)}{r} &= \sqrt{\frac{2(2\nu)^{l+3/2}(k!)}{\Gamma(k+l+3/2)}}r^l e^{-\nu r^2}L_k^{(l+1/2)}(2\nu r^2) \\
&= \sqrt{\frac{2(k!)}{b^3\Gamma(k+l+3/2)}}(\frac{r}{b})^l e^{-\frac{1}{2}(\frac{r}{b})^2}L_k^{(l+1/2)}(\frac{r^2}{b^2}), \quad 2\nu = \frac{1}{b^2},
\end{aligned}$$

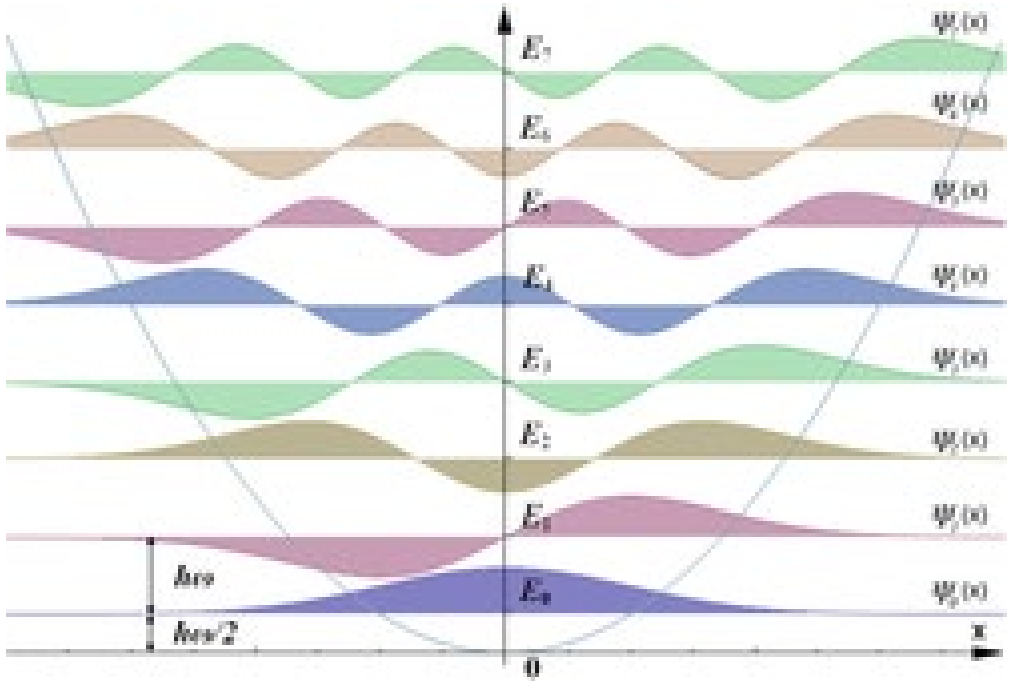
HO basis

- H.O. basis is ortho-normal/ complete.
- Good basis for bound state wave function:
  exponential fall off at large r
- Not good for continuum wave function
- Analytic matrix elements available
- Exact separation of C.M. motion in many-body w.f.

$$\int_0^\infty dx L_n^{(\alpha)}(x) L_m^{(\alpha)}(x) e^{-x} x^\alpha = \frac{\Gamma(n+\alpha+1)}{n!}\delta_{nm}.$$

$$\int_0^\infty dr r^2 (\frac{u_{nl}(r)}{r})(\frac{u_{ml}(r)}{r}) = \delta_{nm}$$

$$E_{kl} = \hbar\omega(N + \frac{3}{2}) = \hbar\omega(2k + l + \frac{3}{2}), \quad n \geq 0, \; k = (N-l)/2 \geq 0$$



HO basis

Matrix elements for HO radial functions:

- You can construct matrix elements in (radial) H.O. basis
- Keep in mind of angular part (spherical harmonics).

$$\langle n, l|O|n', l'\rangle \equiv \int dr r^2 \frac{u_{nl}}{r} \hat{O} \frac{u_{n'l'}}{r}$$

Examples of matrix elements : Not going to prove these relations

$$\langle n, l+1|r|n, l\rangle = b\sqrt{n+l+3/2},$$
$$\langle n-1, l+1|r|n, l\rangle = -b\sqrt{n},$$
$$\langle n, l|r^{2m+1}|n, l\rangle = 0 \text{ for any integer m.}$$

$$\langle nl|r^2|nl\rangle = b^2(2n+l+3/2),$$
$$\langle n+1, l|r^2|nl\rangle = -b^2\sqrt{(n+1)(n+l+3/2)},$$
$$\langle n, l+2|r^2|nl\rangle = b^2\sqrt{(n+l+3/2)(n+l+5/2)},$$
$$\langle n-1, l+2|r^2|nl\rangle = -b^2\sqrt{2n(2n+2l+3)},$$

HO basis

Matrix elements for HO radial functions

$$\langle n,l|p^2|n,l\rangle = \frac{\hbar^2}{b^2}(2n+l+3/2),$$

$$\langle n+1,l|p^2|n,l\rangle = \frac{\hbar^2}{b^2}\sqrt{(n+1)(n+l+3/2)},$$

$$\langle n,l+2|p^2|n,l\rangle = -\frac{\hbar^2}{b^2}\sqrt{(n+l+3/2)(n+l+5/2)},$$

$$\langle n-1,l+2|p^2|n,l\rangle = \frac{\hbar^2}{b^2}\sqrt{2n(2n+2l+3)}.$$

$$\langle nl|p^2|ml\rangle = \frac{1}{b^2}(2n+l+\frac{3}{2})\delta_{nm} - \frac{1}{b^4}\langle nl|r^2|ml\rangle$$

$$\begin{aligned}
\hat{T}_l &= -\frac{1}{2\mu}\frac{d^2}{dr^2} + \frac{l(l+1)}{2\mu r^2}, \\
T_{nn} &= \langle nl|\frac{p^2}{2\mu}|nl\rangle = \frac{\hbar^2}{2\mu b^2}(2n+l+3/2), \\
T_{n+1,n} &= T_{n,n+1} = \langle n+1,l|\frac{p^2}{2\mu}|n,l\rangle = \frac{\hbar^2}{2\mu b^2}\sqrt{(n+1)(n+l+3/2)}, \\
T_{n-1,n} &= T_{n,n-1} = \frac{\hbar^2}{2\mu b^2}\sqrt{n(n+l+1/2)}
\end{aligned}$$

Woods-Saxon potential example

Let us consider Woods-Saxon potential.

For simplicity, let us compute potential matrix elements by simple trapezoidal integration.

$$\langle n, l|O|n', l'\rangle \equiv \int dr r^2 \frac{u_{nl}}{r} \hat{O} \frac{u_{n'l'}}{r}$$

```python
def compute_potential_matrix(r, basis_funcs, V_func):
    """
    Compute potential matrix elements  V_{ij} = <phi_i | V | phi_j>
    by numerical integration (trapezoidal rule).
    """
    N_basis = len(basis_funcs)
    V_vals  = V_func(r)
    V_mat   = np.zeros((N_basis, N_basis))

    for i in range(N_basis):
        for j in range(i, N_basis):
            Vij = np.trapezoid(basis_funcs[i] * V_vals * basis_funcs[j], r)
            V_mat[i, j] = Vij
            V_mat[j, i] = Vij

    return V_mat
```

Because we already know exact solution for HO basis,

$$\begin{aligned}\langle \phi_i|H|\phi_j\rangle &= \langle \phi_i|T + V + V_{HO} - V_{HO}|\phi_j\rangle \\ &= E_i^{HO}\delta_{ij} + \langle \phi_i|V - V_{HO}|\phi_j\rangle\end{aligned}$$

Woods-Saxon potential example

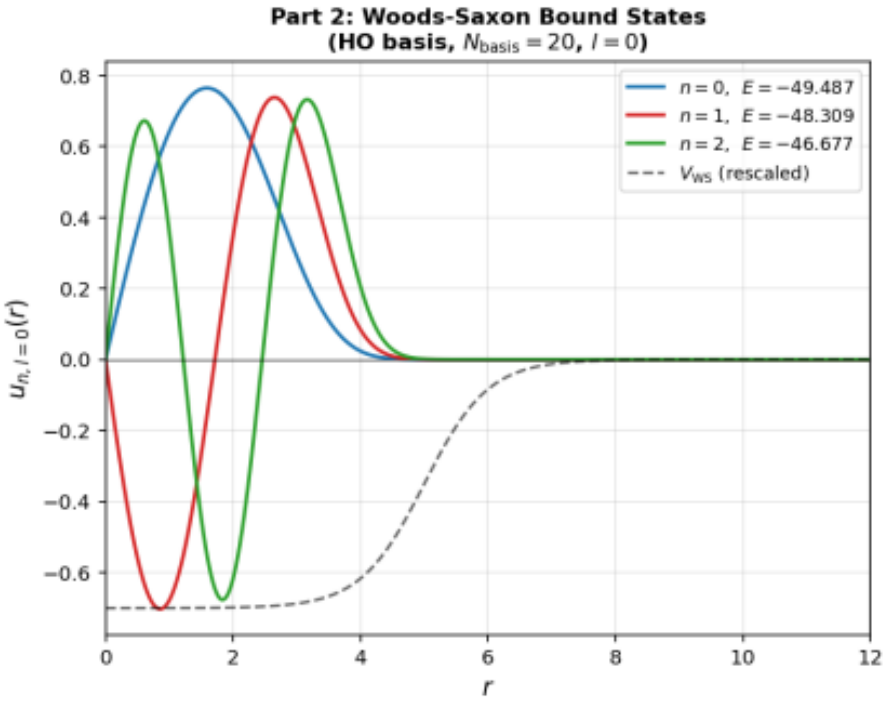
$$
\begin{aligned}
\langle \phi_i | H | \phi_j \rangle &= \langle \phi_i | T + V + V_{HO} - V_{HO} | \phi_j \rangle \\
&= E_i^{HO} \delta_{ij} + \langle \phi_i | V - V_{HO} | \phi_j \rangle
\end{aligned}
$$

```
r = np.linspace(1e-6, r_max, N_grid)

# Build orthonormal HO basis
basis = np.array([exact_ho_wavefunction(r, n, l) for n in range(N_basis)])

# Diagonal: HO kinetic + centrifugal eigenvalues
HO_energies = np.array([exact_ho_energy(n, l) for n in range(N_basis)])

# Off-diagonal: residual potential V_ext - V_HO
V_ext_mat = compute_potential_matrix(r, basis, V_ext_func)
V_ho_mat  = compute_potential_matrix(r, basis, V_harmonic_oscillator)

H = np.diag(HO_energies) + (V_ext_mat - V_ho_mat)

E_vals, E_vecs = eigh(H)
```

Woods-Saxon potential example

V=50; R=5; a=0.5

```
==========================================================================
 PART 2: HO Basis Expansion — Woods-Saxon Potential (l=0)
==========================================================================
 N_basis            E0            E1            E2
-------------------------------------------------
       5     -49.486825    -48.306064    -46.637407
       8     -49.486945    -48.309259    -46.676611
      10     -49.486948    -48.309306    -46.676917
      12     -49.486948    -48.309312    -46.676967
      15     -49.486948    -48.309313    -46.676976
      20     -49.486948    -48.309313    -46.676977
      25     -49.486948    -48.309313    -46.676977
      30     -49.486948    -48.309313    -46.676977
```



We can see a good convergence
with small number of basis (N~ 20)